



SHIVALIK COLLEGE OF ENGINEERING

Recognised under UGC Section (2f) of the UGC Act 1956



A SYNOPSIS OF MINOR PROJECT SUBMITTED AS A PART OF COURSE
CURRICULUM FOR

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING
(Specialization: Cyber Security)

EVIDENCE-FIRST ADAPTIVE AI-SOC
An Explainable Threat Detection, Investigation and
Risk Scoring Platform

Submitted by:

[Student Name 1] — University Roll No.: [•]

[Student Name 2] — University Roll No.: [•]

[Student Name 3] — University Roll No.: [•]

[Student Name 4] — University Roll No.: [•]

Supervisor Name: [•]

Designation: [•]

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SHIVALIK COLLEGE OF ENGINEERING, DEHRADUN
October, 2026

CONTENTS

1. Introduction
2. Problem Statement
3. Objectives of the Project
4. Scope of the Project
5. Literature Review / Existing System
6. Proposed System / Proposed Methodology
7. Tools and Technologies Used
8. System Requirements
9. Methodology / Work Flow
10. Expected Outcomes
11. Future Scope
12. References

1. Introduction

Security Operations Centers (SOCs) are responsible for continuously monitoring an organisation's IT infrastructure, detecting malicious activity and coordinating an appropriate response. As enterprise environments grow, the volume of security telemetry generated by endpoints, servers and network devices has increased dramatically, and analysts are routinely expected to review thousands of alerts every day. Modern approaches to this problem — Security Information and Event Management (SIEM), Extended Detection and Response (XDR), User and Entity Behaviour Analytics (UEBA) and, increasingly, AI-assisted analysis — attempt to reduce this burden, but many AI-driven security tools present conclusions without clearly showing the evidence or reasoning behind them.

The proposed Minor Project, titled “Evidence-First Adaptive AI-SOC: An Explainable Threat Detection, Investigation and Risk Scoring Platform,” is an educational and research-oriented SOC platform that automatically collects security telemetry from endpoints, detects and correlates suspicious activity, enriches indicators using threat intelligence, calculates an explainable risk score, and uses a Large Language Model (LLM) to assist an analyst in investigating incidents.

Rather than simply labelling an activity as malicious, the system is designed to show what happened, which events were observed, why the activity was considered suspicious, which evidence supports or contradicts that conclusion, how the resulting risk score was calculated, which MITRE ATT&CK techniques may be involved, and what an analyst should investigate next. The project brings together concepts from SIEM, EDR/XDR, threat intelligence, detection engineering, UEBA, machine learning, risk scoring, incident correlation, Generative AI/LLMs, Retrieval-Augmented Generation (RAG), SOAR-style response workflows, MITRE ATT&CK and threat hunting into a single, human-supervised platform.

2. Problem Statement

Modern endpoints generate extremely large numbers of security events — a single endpoint may produce tens of thousands of raw log entries during normal daily activity. A SOC analyst cannot be expected to manually examine every such event, yet critical indicators of compromise are often buried within this volume of largely benign activity.

Two related problems motivate this project:

1. High alert volume and weak prioritisation: Existing monitoring pipelines frequently forward every generated event or alert to the analyst instead of transforming raw events into a small number of prioritised, correlated incidents that genuinely warrant investigation.
2. Lack of explainability in AI-based security tools: AI-based detection and triage systems can generate plausible-sounding explanations without clearly demonstrating why a particular conclusion was reached, which evidence supports it, or what evidence would be required to increase confidence. This reduces analyst trust and makes it difficult to validate AI-generated conclusions before acting on them.

The proposed system addresses both problems by transforming Raw Events → Suspicious Events → Detections → Correlated Incidents → Prioritised Investigation, and by following an Evidence-First AI approach in which every significant AI conclusion is, wherever possible, linked back to concrete security evidence.

3. Objectives of the Project

The Minor Project aims to achieve the following objectives:

3. To design and implement a lightweight endpoint collector that automatically gathers security telemetry (process, authentication, network and log data) from Windows, Linux and macOS endpoints.
4. To build a normalization layer that converts heterogeneous operating-system event formats into a common event schema for unified detection and querying.
5. To implement a multi-layered detection engine combining rule-based detection (Sigma/custom rules), IOC-based detection using threat-intelligence feeds, behavioural sequence analysis, and machine-learning-based anomaly detection.
6. To design a dynamic and explainable risk-scoring mechanism in which every risk score can be decomposed into the measurable factors that produced it, and in which the score evolves transparently as new evidence is discovered.
7. To develop an event correlation engine and an evidence graph that group related alerts into a single, well-supported incident rather than multiple disconnected alerts.

8. To integrate an LLM-based AI SOC Investigator that operates on structured incident context, separates observed facts from system inferences and AI hypotheses, and produces a threat hypothesis alongside a benign counter-hypothesis.
9. To incorporate a Retrieval-Augmented Generation (RAG) pipeline so that AI explanations are grounded in trusted cybersecurity knowledge such as MITRE ATT&CK documentation and incident-response playbooks.
10. To build a human-supervised response and analyst-feedback loop in which recommended containment actions require analyst approval and analyst feedback (True Positive / False Positive / Benign Positive) is used to propose and validate improved detection rules through historical simulation before deployment.

4. Scope of the Project

The scope of the project covers the design, development and demonstration of a research-oriented AI-SOC platform within the duration of the Minor Project. The following areas are included within scope:

- Automatic collection of endpoint telemetry from Windows, Linux and macOS sources (Sysmon, Windows Event Logs, journald, auth.log, auditd, macOS Unified Logs).
- Rule-based, IOC-based and behaviour/ML-based detection, threat-intelligence enrichment (VirusTotal, AbuseIPDB), event correlation and dynamic, explainable risk scoring.
- An evidence graph and MITRE ATT&CK mapping to visually and technically connect related events within an incident.
- An AI investigation layer (LLM + RAG) that produces plain-English explanations, dual (threat/benign) hypotheses, and identifies missing evidence.
- A natural-language and simple custom-query threat-hunting interface, presented alongside a SOC dashboard for incident review.
- A human-supervised response workflow and an analyst-feedback-driven detection-improvement loop, validated through historical rule simulation.

The following are outside the scope of the current Minor Project and are identified instead as future work:

- Enterprise-scale deployment, high-availability infrastructure and support for network-level sources such as firewalls, IDS/IPS and cloud platforms (Microsoft 365, IaaS/PaaS telemetry).
- Fully automated (unsupervised) response actions and production-grade local/hybrid LLM deployment with data-redaction guarantees.
- Competing with the scale, global threat-intelligence coverage, or certified reliability of commercial SIEM/XDR products.

5. Literature Review / Existing System

Commercial and open-source security-monitoring platforms already implement several of the fundamental capabilities required for SOC operations, including telemetry collection, centralized event analysis, detection rules, threat-intelligence enrichment, event correlation, risk prioritisation, MITRE ATT&CK mapping, behavioural analysis and, increasingly, AI-assisted triage.

5.1 SIEM and XDR Platforms

Security Information and Event Management (SIEM) systems such as Splunk and Microsoft Sentinel, and Extended Detection and Response (XDR) platforms, aggregate logs from multiple sources and apply correlation rules to raise alerts. Open-source alternatives such as the Elastic (ELK) stack, OpenSearch-based SIEMs and Wazuh provide similar capabilities with lower licensing cost, and are commonly used as a reference architecture for telemetry storage and search.

5.2 Detection Engineering and Threat Intelligence

Sigma has emerged as a widely adopted, platform-independent format for writing detection rules, allowing the same logical rule to be translated into multiple SIEM query languages. Threat-intelligence services such as VirusTotal and AbuseIPDB provide reputation data for indicators of compromise (IPs, domains, hashes) and are commonly integrated into detection pipelines for automatic enrichment.

5.3 UEBA and Machine-Learning-Based Detection

User and Entity Behaviour Analytics (UEBA) approaches build behavioural baselines for users and endpoints and flag statistically significant deviations, often using unsupervised algorithms

such as Isolation Forest or clustering. These approaches complement signature-based detection by identifying previously unseen ('unknown-unknown') attack patterns, though they typically require careful tuning to control false-positive rates.

5.4 AI/LLM-Assisted Security Operations

More recently, generative AI and LLMs have been explored for SOC use cases such as alert summarisation, natural-language threat hunting, and automated report generation. However, published discussion of these tools frequently highlights a key limitation: AI-generated security conclusions are not always traceable to specific supporting evidence, which limits analyst trust and complicates verification, particularly in regulated or high-stakes environments.

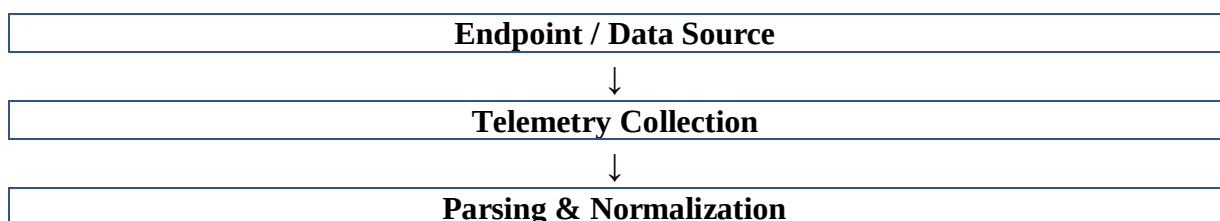
5.5 Research Gap

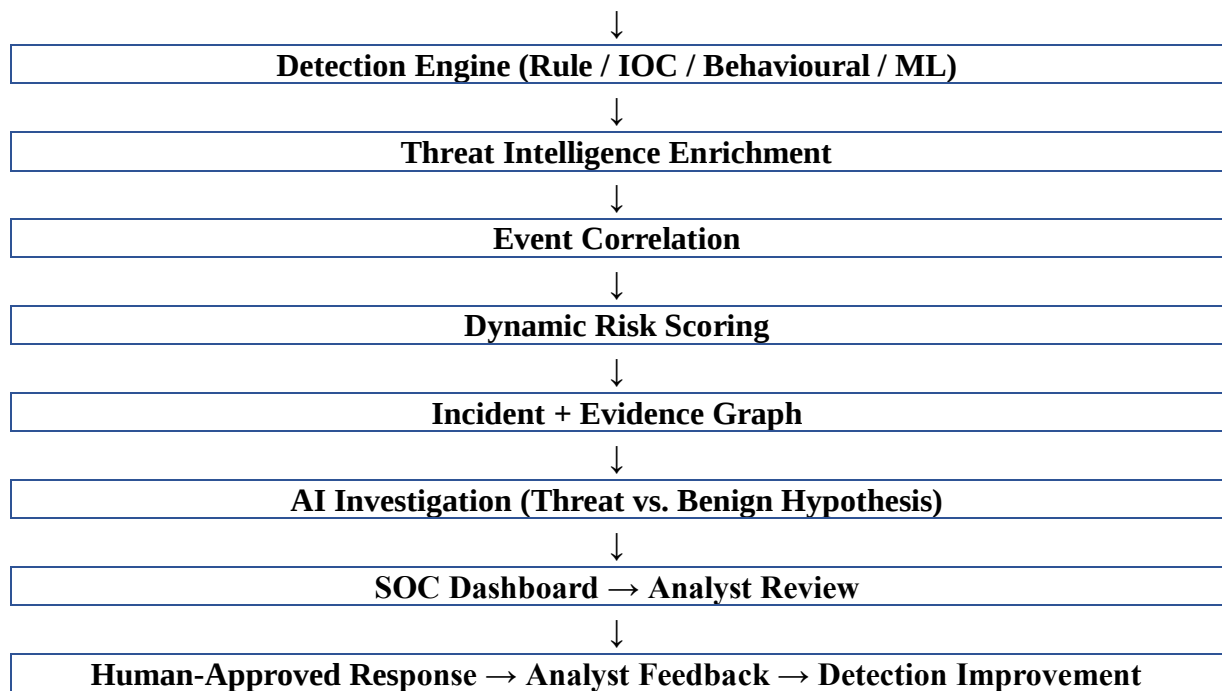
Existing systems generally combine some subset of detection, correlation, risk scoring and AI assistance, but few explicitly separate observed facts, system inferences and AI hypotheses, or explicitly evaluate a benign counter-hypothesis alongside a threat hypothesis. Similarly, AI-suggested detection-rule improvements are rarely validated against historical telemetry before being proposed to an analyst. This gap — transparent, evidence-linked, human-supervised AI investigation and detection improvement — is the specific focus of the proposed project. An initial proof-of-concept prototype has already been built by the team, incorporating macOS process monitoring, VirusTotal/AbuseIPDB enrichment, basic MITRE mapping, a Streamlit dashboard and a FastAPI/SQLite backend, confirming the technical feasibility of the overall approach and forming the starting point for the proposed Version 2 architecture.

6. Proposed System / Proposed Methodology

The proposed system follows the design philosophy: Observe → Detect → Correlate → Enrich → Score → Prove → Challenge → Explain → Decide → Learn. Detection engines identify suspicious activity, evidence supports each conclusion, AI investigates and explains the evidence, and the human analyst retains control over important decisions.

6.1 End-to-End Workflow





6.2 Detection Engine

The detection engine combines four complementary techniques rather than relying on any single approach:

- Rule-based detection using Sigma and custom rules for known suspicious patterns (e.g., an office application spawning PowerShell with an encoded command).
- IOC-based detection, checking extracted IPs, domains, URLs and file hashes against VirusTotal, AbuseIPDB and other threat-intelligence feeds, with local caching to avoid redundant lookups.
- Behavioural detection, which evaluates sequences and relationships between events (e.g., Word → PowerShell → Encoded Command → Download → Suspicious Connection) rather than judging individual processes in isolation.
- Machine-learning/anomaly detection (Isolation Forest, PyOD, clustering) over features such as login frequency/time, process frequency and new destinations, feeding into risk scoring rather than issuing independent verdicts.

6.3 Explainable, Dynamic Risk Scoring

Every incident receives a risk score built from measurable, individually visible factors, so that an analyst asking “why is this incident risk 92?” can see the complete breakdown. An illustrative scoring model is shown below.

Risk Factor	Example Contribution
Threat-intelligence reputation	+25
Suspicious process chain	+20
Behavioural anomaly	+15
Authentication anomaly	+15
Event correlation	+10
Asset importance	+10
Historical context	+5

(The exact weighting will be designed and experimentally validated during development.) The score is recalculated as new evidence is discovered — for instance, rising from an initial 32 to 91 as an encoded command, a suspicious destination, poor threat-intelligence reputation and cross-endpoint recurrence are successively confirmed — so the analyst can see exactly which evidence drove each change.

6.4 Evidence Graph and MITRE ATT&CK Mapping

An evidence graph (built using NetworkX and visualised with Cytoscape.js/D3.js) links entities such as users, endpoints, processes and network indicators, showing why individual events belong to the same investigation rather than presenting isolated alerts. Detected behaviour is additionally mapped to relevant MITRE ATT&CK tactics and techniques so the analyst can relate observed activity to known adversary patterns.

6.5 AI SOC Investigator

An LLM operates on structured incident context (events, detections, risk breakdown, process tree, threat intelligence, MITRE mapping and the evidence graph) rather than raw logs, and produces a plain-English explanation, investigation recommendations, and suggested containment/remediation steps. Its output is explicitly separated into:

- Observed Facts — directly supported by telemetry (e.g., 15 failed logins, PowerShell executed).
- System Inference — derived from detection/behavioural logic (e.g., anomaly score = 87).
- AI Hypothesis — a proposed explanation clearly presented as a hypothesis rather than a confirmed fact.

The investigator additionally evaluates a Threat Hypothesis and a Benign Counter-Hypothesis side by side (for example, weighing a possible credential compromise against the possibility of an authorised administrative action), and identifies missing evidence — such as persistence status or file-approval records — that would increase or decrease confidence, optionally triggering controlled follow-up searches.

6.6 Retrieval-Augmented Generation (RAG) and Threat Hunting

A RAG pipeline (MITRE ATT&CK documentation, detection notes and IR playbooks chunked, embedded and stored in a vector database such as Qdrant or Chroma) grounds AI explanations in trusted reference material. A progressive threat-hunting interface allows an analyst to move from natural language (“Show suspicious PowerShell activity today”) to a simple custom query language (HUNT powershell WHERE risk > 70 LAST 24h) and, ultimately, to the underlying structured query — supporting both investigation and analyst learning.

6.7 Human-Supervised Response and Self-Improving Detection

Recommended response actions (kill process, block IP, isolate endpoint, disable account) require analyst approval before execution. After an investigation, the analyst labels the incident as True Positive, False Positive, Benign Positive or Needs Investigation; this feedback is analysed to identify detection rules producing excessive false positives, and any AI-proposed rule modification is tested against historical telemetry — comparing old vs. proposed detection/false-positive rates — before an analyst approves, modifies or rejects the change.

7. Tools and Technologies Used

Category	Tools / Technologies
Programming Language	Python
Endpoint Telemetry	Sysmon, Windows Event Logs, psutil, macOS Unified Logs, Linux auditd/journald
Backend	FastAPI
Telemetry Storage	OpenSearch
Application Database	PostgreSQL (SQLite in current prototype)
Optional Cache	Redis
Detection Engine	Sigma, custom Python rule engine, custom behavioural rules
Machine Learning	scikit-learn, PyOD, Pandas, NumPy

Threat Intelligence	VirusTotal API, AbuseIPDB API
Threat Framework	MITRE ATT&CK / STIX data
Graph & Visualization	NetworkX; Cytoscape.js / D3.js
AI / LLM	Cloud LLM API and/or local LLM via Ollama (model-independent design)
RAG	LlamaIndex / LangChain; Qdrant or Chroma vector database
Frontend	React / Next.js (Streamlit for rapid prototyping)
Real-Time Communication	WebSockets
Deployment	Docker, Docker Compose
Development & Version Control	Git, GitHub, VS Code

8. System Requirements

8.1 Hardware Requirements

- Processor: Intel Core i5 (10th Gen) / AMD Ryzen 5 or higher (per development machine).
- RAM: Minimum 8 GB; 16 GB or higher recommended for running local ML models, OpenSearch and containerised services simultaneously.
- Storage: Minimum 20 GB free disk space (SSD recommended) for telemetry data, containers and vector-database indices.
- Network: Stable internet connection for threat-intelligence API calls (VirusTotal, AbuseIPDB) and, where used, cloud-LLM access.
- Optional: GPU-enabled machine for locally hosted LLM inference (via Ollama), if cloud LLM access is not used.

8.2 Software Requirements

- Operating System: Windows 10/11, Linux (Ubuntu 20.04+) or macOS, for both development and endpoint-agent testing.
- Programming Language and Runtime: Python 3.10+, Node.js (for the React/Next.js frontend).
- Frameworks/Libraries: FastAPI, scikit-learn, PyOD, Pandas, NumPy, NetworkX, LangChain/LlamaIndex.

- Databases: OpenSearch, PostgreSQL (SQLite for prototyping), Qdrant/Chroma vector database, optional Redis.
- Containerisation: Docker and Docker Compose.
- Version Control and IDE: Git, GitHub, Visual Studio Code.
- Detection Tooling: Sigma rule compiler, Sysmon configuration files.

9. Methodology / Work Flow

The project will be developed incrementally over five phases, each building on the previous phase's output.

Phase 1 — Foundation

Build the endpoint telemetry collector, normalization layer and event storage, together with a basic dashboard. Goal: automatically collect and search real security telemetry.

Phase 2 — Detection

Add Sigma/custom rules, IOC detection, threat-intelligence enrichment, behavioural rules, event correlation and risk scoring, generating prioritised incidents. Goal: turn large numbers of raw events into a small number of prioritised security incidents.

Phase 3 — AI Investigation

Integrate the LLM and RAG pipeline to produce plain-English, evidence-backed explanations and MITRE-technique explanations. Goal: create the AI SOC investigation layer.

Phase 4 — Advanced Investigation

Add the evidence graph, dynamic risk evolution, dual (threat/benign) hypotheses, evidence-seeking investigation, natural-language hunting, the custom query language and UEBA-lite baselining. Goal: build the platform's main differentiating capabilities.

Phase 5 — Response and Learning

Add human-approved response actions, the analyst-feedback system, false-positive analysis, AI-proposed detection-rule improvement, historical rule simulation, rule versioning and a teaching mode. Goal: complete the human-supervised, adaptive SOC loop.

Illustrative End-to-End Scenario

A representative test scenario — 15 failed logins followed by a successful login, WINWORD.EXE launching PowerShell with an encoded command, a file download, and a connection to a suspicious IP — will be used to validate the complete pipeline: automatic collection → normalization → detection → threat-intelligence enrichment → behavioural analysis → correlation into a single incident → risk scoring → evidence-graph construction → AI investigation (facts, inference, threat/benign hypotheses, missing evidence, MITRE mapping, recommended actions) → analyst review and feedback.

10. Expected Outcomes

By the end of the project, the team expects to deliver a working, end-to-end demonstration of an evidence-first, explainable AI-SOC pipeline, including:

11. A functional endpoint collector and normalization layer capable of ingesting telemetry from at least one operating system in real time.
12. A multi-technique detection engine (rule-based, IOC-based, behavioural and ML-based) producing prioritised, correlated incidents rather than raw alert floods.
13. An explainable, dynamic risk-scoring module in which every score can be traced back to its contributing factors and updates visibly as new evidence arrives.
14. An evidence graph and MITRE ATT&CK mapping for each generated incident.
15. An AI investigation module that produces fact/inference/hypothesis-separated, RAG-grounded explanations, including a threat hypothesis and a benign counter-hypothesis for each incident.
16. A working SOC dashboard supporting natural-language and simple-query threat hunting, human-approved response actions and analyst feedback capture.
17. A demonstrated human-supervised detection-improvement loop, in which at least one AI-proposed rule change is validated through historical simulation before acceptance.
18. Project documentation, source code, and a final presentation/demonstration of the complete attack-to-response scenario described in Section 9.

11. Future Scope

Beyond the current Minor Project, the architecture is designed to be extended in several directions:

- Additional telemetry sources: firewalls, IDS/IPS, routers, cloud platforms, Microsoft 365, web servers, identity providers and other security products.
- Alternative/additional collectors such as Fluent Bit, Vector, Winlogbeat or the Wazuh Agent for production-scale deployments.
- A graph database (e.g., Neo4j) for the evidence graph as incident volume and relationship complexity grow.
- Privacy-aware AI operation, including local-LLM (Ollama) and hybrid cloud/local modes with configurable redaction of sensitive fields (usernames, internal IPs, hostnames) before information is sent to an external LLM.
- MITRE ATT&CK coverage visualisation to help identify tactics/techniques for which telemetry or detection coverage is currently weak.
- Expansion of the custom security query language and progressive threat-hunting interface toward closer parity with SPL/KQL-style capabilities.
- Broader analyst “teaching mode” content to support cybersecurity/SOC learning beyond the scope of this project.

12. References

- [1] MITRE ATT&CK®, “Enterprise Matrix,” The MITRE Corporation. [Online]. Available: <https://attack.mitre.org/>
- [2] SigmaHQ, “Sigma — Generic Signature Format for SIEM Systems,” GitHub Repository. [Online]. Available: <https://github.com/SigmaHQ/sigma>
- [3] OpenSearch Project, “OpenSearch Documentation,” Amazon Web Services. [Online]. Available: <https://opensearch.org/>
- [4] VirusTotal, “VirusTotal API Documentation,” Google LLC. [Online]. Available: <https://developers.virustotal.com/>
- [5] AbuseIPDB, “AbuseIPDB API Documentation.” [Online]. Available: <https://docs.abuseipdb.com/>
- [6] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

- [7] Y. Zhao, Z. Nasrullah, and Z. Li, “PyOD: A Python Toolbox for Scalable Outlier Detection,” *Journal of Machine Learning Research*, vol. 20, no. 96, pp. 1–7, 2019.
- [8] Wazuh Inc., “Wazuh — The Open Source Security Platform,” Documentation. [Online]. Available: <https://documentation.wazuh.com/>
- [9] LangChain, “LangChain Documentation — Retrieval-Augmented Generation.” [Online]. Available: <https://python.langchain.com/>
- [10] Qdrant, “Qdrant Vector Database Documentation.” [Online]. Available: <https://qdrant.tech/documentation/>
- [11] Microsoft, “Sysmon — System Monitor,” Sysinternals Documentation. [Online]. Available: <https://learn.microsoft.com/sysinternals/downloads/sysmon>